```
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).
```

Alternative to `drive.mount()`: Using `PyDrive`

If `drive.mount()` is not working or if you prefer to interact with Google Drive files programmatically rather than mounting it as a filesystem, you can use the `PyDrive` library. This allows you to authenticate and then list, upload, or download files directly.

```
# Install the PyDrive library
!pip install -U -q PyDrive
```

```
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials

# Authenticate Colab user (this will open a new window for authentication if needed)
auth.authenticate_user()

# Create GoogleAuth instance.
gauth = GoogleAuth()

# Explicitly set 'embedded' to True to prevent PyDrive from attempting
# to launch a local webserver for authentication, which requires client_secrets.json.
# This forces PyDrive to use an authentication flow suitable for Colab.
gauth.embedded = True

# Explicitly set 'client_config_backend' in gauth's settings AFTER initialization
# as a defensive measure against other potential client config loads.
gauth.settings['client_config_backend'] = 'settings'

# Load the credentials obtained from google.colab.auth
gauth.credentials = GoogleCredentials.get_application_default()

# Create the PyDrive client using the gauth object
drive = GoogleDrive(gauth)

print('PyDrive client authenticated and ready.')
```

```
PyDrive client authenticated and ready.
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)


from google.colab import files
uploaded = files.upload()


import pandas as pd
import io
df = pd.read_csv(io.BytesIO(list(uploaded.values())[0]))
print(df.shape)
```

```
Mounted at /content/drive
Choose Files tested.csv
tested.csv(text/csv) - 29474 bytes, last modified: 6/16/2026 - 100% done
Saving tested.csv to tested.csv
(418, 12)
```

```
from pydrive.auth import GoogleAuth
from pydrive.drive import GoogleDrive
from google.colab import auth
from oauth2client.client import GoogleCredentials

# Re-authenticate Colab user (ensures credentials are fresh and available)
auth.authenticate_user()
```

```
# Create GoogleAuth instance.
gauth = GoogleAuth()

# Load the credentials obtained from google.colab.auth
# This is the correct way to link Colab authentication with PyDrive
gauth.credentials = GoogleCredentials.get_application_default()

# Create the PyDrive client using the gauth object
drive = GoogleDrive(gauth)

# Example: List files in your Google Drive root directory
# You can adjust the 'q' parameter to search for specific files/folders
file_list = drive.ListFile({'q': "'root' in parents and trashed=false"}).GetList()
for file in file_list:
  print('title: %s, id: %s' % (file['title'], file['id']))

print(f'\nFound {len(file_list)} files/folders in your Drive root.')
```

```
from google.colab import drive
drive.mount('/content/drive', force_remount=True)
```

```
Mounted at /content/drive
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files tested.csv

tested.csv(text/csv) - 29474 bytes, last modified: 6/16/2026 - 100% done

```
Saving tested.csv to tested (1).csv
```

```
import pandas as pd
import io
df = pd.read_csv(io.BytesIO(list(uploaded.values())[0]))
print(df.shape)
```

```
(418, 12)
```

```
print(df.columns)
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
      dtype='object')
```

```
# Q#1 Definitions

# 1. NumPy
print("1. NumPy:")
print("NumPy is a Python library used for numerical computations. It supports arrays, matrices, and mathematical functions.")
print()

# 2. Matplotlib
print("2. Matplotlib:")
print("Matplotlib is a Python library used for data visualization. It creates charts, graphs, and plots.")
print()

# 3. Pandas
print("3. Pandas:")
print("Pandas is a Python library used for data manipulation and analysis. It works with tables called DataFrames.")
print()

# 4. TensorFlow
print("4. TensorFlow:")
print("TensorFlow is a Python library by Google used for deep learning and building neural networks.")
print()

# 5. Seaborn
print("5. Seaborn:")
print("Seaborn is a Python library for statistical data visualization. It makes beautiful charts easily.")
print()

# 6. Sklearn
print("6. Sklearn (Scikit-learn):")
print("Sklearn is a Python library for machine learning. It has tools for classification, regression, and clustering.")
```

```
1. NumPy:
NumPy is a Python library used for numerical computations. It supports arrays, matrices, and mathematical functions.

2. Matplotlib:
Matplotlib is a Python library used for data visualization. It creates charts, graphs, and plots.

3. Pandas:
Pandas is a Python library used for data manipulation and analysis. It works with tables called DataFrames.

4. TensorFlow:
TensorFlow is a Python library by Google used for deep learning and building neural networks.

5. Seaborn:
Seaborn is a Python library for statistical data visualization. It makes beautiful charts easily.

6. Sklearn (Scikit-learn):
Sklearn is a Python library for machine learning. It has tools for classification, regression, and clustering.
```

```
# Q#2 Dataset Analysis

# Number of features (columns)
print("Number of Features (columns):", df.shape[1])
print()

# Number of instances (rows)
print("Number of Instances (rows):", df.shape[0])
print()

# List at least 5 features
print("5 Features of Titanic Dataset:")
print("1.", df.columns[0])
print("2.", df.columns[1])
print("3.", df.columns[2])
print("4.", df.columns[3])
print("5.", df.columns[4])
```

```
Number of Features (columns): 12

Number of Instances (rows): 418

5 Features of Titanic Dataset:
1. PassengerId
2. Survived
3. Pclass
4. Name
5. Sex
```

```
# Dataset Information

print("1. Website Name of Dataset:")
print("   Kaggle - www.kaggle.com")
print()

print("2. Is Titanic a Benchmark Dataset?")
print("   YES! Titanic is a very famous Benchmark dataset.")
print("   It is used worldwide for learning Machine Learning.")
print()

print("3. Is data Structured or Unstructured?")
print("   STRUCTURED data - because it is in table format (rows and columns)")
print()

print("4. Check if any Text in cells:")
print(df.dtypes)
```

```
1. Website Name of Dataset:
   Kaggle - www.kaggle.com

2. Is Titanic a Benchmark Dataset?
   YES! Titanic is a very famous Benchmark dataset.
   It is used worldwide for learning Machine Learning.

3. Is data Structured or Unstructured?
   STRUCTURED data - because it is in table format (rows and columns)

4. Check if any Text in cells:
PassengerId      int64
Survived         int64
Pclass           int64
Name            object
Sex             object
Age            float64
SibSp            int64
Parch            int64
Ticket          object
Fare           float64
Cabin           object
Embarked        object
dtype: object
```

```
# ML Process Steps

print("Machine Learning Process:")
print()
print("1. Data Collection")
print("2. Data Preprocessing")
print("3. Exploratory Data Analysis (EDA)")
print("4. Feature Selection")
print("5. Model Selection")
print("6. Model Training")
print("7. Model Testing")
print("8. Model Evaluation")
print("9. Model Deployment")
```

```
Machine Learning Process:

1. Data Collection
2. Data Preprocessing
3. Exploratory Data Analysis (EDA)
4. Feature Selection
5. Model Selection
6. Model Training
7. Model Testing
8. Model Evaluation
9. Model Deployment
```

```
# Prepare data for KNN Model
from sklearn.preprocessing import LabelEncoder

# Drop text columns that can't be used
df_model = df.drop(['Name', 'Ticket', 'Cabin'], axis=1)

# Fill missing values
df_model = df_model.fillna(df_model.mean(numeric_only=True))
df_model['Embarked'] = df_model['Embarked'].fillna('S')
```

```
# Convert text to numbers
le = LabelEncoder()
df_model['Sex'] = le.fit_transform(df_model['Sex'])
df_model['Embarked'] = le.fit_transform(df_model['Embarked'])

print("Data Prepared Successfully!")
print(df_model.shape)
```

```
Data Prepared Successfully!
(418, 9)
```

```
# Train KNN Model
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

# Define X (features) and y (target)
X = df_model.drop('Survived', axis=1)
y = df_model['Survived']

# Split data into training and testing
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Create KNN model
knn = KNeighborsClassifier(n_neighbors=5)

# Train the model
knn.fit(X_train, y_train)

print("KNN Model Trained Successfully!")
print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

```
KNN Model Trained Successfully!
Training size: (334, 8)
Testing size: (84, 8)
```

```
print("Today  task  23 June 2026");
```

```
Today  task  23 June 2026
```

```
# Simple Linear Regression Model
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create Linear Regression model
lr = LinearRegression()

# Train the model
lr.fit(X_train, y_train)

# Training Results
y_train_pred = lr.predict(X_train)
train_mse = mean_squared_error(y_train, y_train_pred)
train_r2 = r2_score(y_train, y_train_pred)

print("=== Training Results ===")
print("MSE:", round(train_mse, 4))
print("R2 Score:", round(train_r2, 4))
print()

# Testing Results
y_test_pred = lr.predict(X_test)
test_mse = mean_squared_error(y_test, y_test_pred)
test_r2 = r2_score(y_test, y_test_pred)

print("=== Testing Results ===")
print("MSE:", round(test_mse, 4))
print("R2 Score:", round(test_r2, 4))
```

```
=== Training Results ===
MSE: 0.0
R2 Score: 1.0

=== Testing Results ===
MSE: 0.0
R2 Score: 1.0
```

```
# Fix - Reload and split data properly
import pandas as pd
import io
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Reload dataset
df2 = pd.read_csv(io.BytesIO(list(uploaded.values())[0]))

# Drop unnecessary columns
df2 = df2.drop(['Name', 'Ticket', 'Cabin'], axis=1)

# Fill missing values
df2 = df2.fillna(df2.mean(numeric_only=True))
df2['Embarked'] = df2['Embarked'].fillna('S')

# Convert text to numbers
le = LabelEncoder()
df2['Sex'] = le.fit_transform(df2['Sex'])
df2['Embarked'] = le.fit_transform(df2['Embarked'])

# Split properly
X2 = df2.drop('Survived', axis=1)
y2 = df2['Survived']

X2_train, X2_test, y2_train, y2_test = train_test_split(X2, y2, test_size=0.2, random_state=42)

print("Data reloaded successfully!")
print("Training size:", X2_train.shape)
print("Testing size:", X2_test.shape)
```

```
Data reloaded successfully!
Training size: (334, 8)
Testing size: (84, 8)
```

```
# Simple Linear Regression Model (Fixed)
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create Linear Regression model
lr = LinearRegression()

# Train the model
lr.fit(X2_train, y2_train)

# Training Results
y2_train_pred = lr.predict(X2_train)
train_mse = mean_squared_error(y2_train, y2_train_pred)
train_r2 = r2_score(y2_train, y2_train_pred)

print("=== Training Results ===")
print("MSE:", round(train_mse, 4))
print("R2 Score:", round(train_r2, 4))
print()

# Testing Results
y2_test_pred = lr.predict(X2_test)
test_mse = mean_squared_error(y2_test, y2_test_pred)
test_r2 = r2_score(y2_test, y2_test_pred)

print("=== Testing Results ===")
print("MSE:", round(test_mse, 4))
print("R2 Score:", round(test_r2, 4))
```

```
=== Training Results ===
MSE: 0.0
R2 Score: 1.0

=== Testing Results ===
MSE: 0.0
R2 Score: 1.0
```

```
# Logistic Regression Model (Correct for Titanic)
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, f1_score, confusion_matrix
```

```
# Create model
log_reg = LogisticRegression(max_iter=1000)

# Train model
log_reg.fit(X2_train, y2_train)

# Training Results
y2_train_pred = log_reg.predict(X2_train)
print("=== Training Results ===")
print("Accuracy:", round(accuracy_score(y2_train, y2_train_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y2_train, y2_train_pred) * 100, 2), "%")
print("F1 Score:", round(f1_score(y2_train, y2_train_pred) * 100, 2), "%")
print()

# Testing Results
y2_test_pred = log_reg.predict(X2_test)
print("=== Testing Results ===")
print("Accuracy:", round(accuracy_score(y2_test, y2_test_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y2_test, y2_test_pred) * 100, 2), "%")
print("F1 Score:", round(f1_score(y2_test, y2_test_pred) * 100, 2), "%")
```

```
=== Training Results ===
Accuracy: 100.0 %
Precision: 100.0 %
F1 Score: 100.0 %

=== Testing Results ===
Accuracy: 100.0 %
Precision: 100.0 %
F1 Score: 100.0 %
```

```
# Load Titanic directly without uploading!
url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
df3 = pd.read_csv(url)
print("Dataset loaded!")
print(df3.shape)
print(df3.head())
```

```
Dataset loaded!
(891, 12)
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
```

```
# Preprocess new dataset
df3 = df3.drop(['Name', 'Ticket', 'Cabin'], axis=1)

# Fill missing values
df3 = df3.fillna(df3.mean(numeric_only=True))
df3['Embarked'] = df3['Embarked'].fillna('S')

# Convert text to numbers
le = LabelEncoder()
df3['Sex'] = le.fit_transform(df3['Sex'])
df3['Embarked'] = le.fit_transform(df3['Embarked'])

# Split data
X3 = df3.drop('Survived', axis=1)
y3 = df3['Survived']

X3_train, X3_test, y3_train, y3_test = train_test_split(X3, y3, test_size=0.2, random_state=42)
```

```
# Train Logistic Regression
log_reg2 = LogisticRegression(max_iter=1000)
log_reg2.fit(X3_train, y3_train)

# Training Results
y3_train_pred = log_reg2.predict(X3_train)
print("=== Training Results ===")
print("Accuracy:", round(accuracy_score(y3_train, y3_train_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y3_train, y3_train_pred) * 100, 2), "%")
print("F1 Score:", round(f1_score(y3_train, y3_train_pred) * 100, 2), "%")
print()

# Testing Results
y3_test_pred = log_reg2.predict(X3_test)
print("=== Testing Results ===")
print("Accuracy:", round(accuracy_score(y3_test, y3_test_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y3_test, y3_test_pred) * 100, 2), "%")
print("F1 Score:", round(f1_score(y3_test, y3_test_pred) * 100, 2), "%")
```

```
=== Training Results ===
Accuracy: 80.62 %
Precision: 76.64 %
F1 Score: 73.05 %

=== Testing Results ===
Accuracy: 81.01 %
Precision: 78.57 %
F1 Score: 76.39 %
```

```
from google.colab import files
uploaded3 = files.upload()
```

Choose Files processed.cleveland.data

processed.cleveland.data(n/a) - 18461 bytes, last modified: 6/23/2026 - 100% done

Saving processed.cleveland.data to processed.cleveland.data

```
import pandas as pd

# Read the .data file
df_heart = pd.read_csv(list(uploaded3.keys())[0], header=None)

# Add column names
df_heart.columns = ['age', 'sex', 'cp', 'trestbps', 'chol',
                    'fbs', 'restecg', 'thalach', 'exang',
                    'oldpeak', 'slope', 'ca', 'thal', 'target']

print("Dataset Loaded!")
print(df_heart.shape)
print(df_heart.head())
```

```
Dataset Loaded!
(303, 14)
    age  sex   cp  trestbps   chol  fbs  restecg  thalach  exang  oldpeak  \
0  63.0  1.0  1.0     145.0  233.0  1.0      2.0    150.0    0.0      2.3
1  67.0  1.0  4.0     160.0  286.0  0.0      2.0    108.0    1.0      1.5
2  67.0  1.0  4.0     120.0  229.0  0.0      2.0    129.0    1.0      2.6
3  37.0  1.0  3.0     130.0  250.0  0.0      0.0    187.0    0.0      3.5
4  41.0  0.0  2.0     130.0  204.0  0.0      2.0    172.0    0.0      1.4

   slope   ca thal  target
0    3.0  0.0  6.0       0
1    2.0  3.0  3.0       2
2    2.0  2.0  7.0       1
3    3.0  0.0  3.0       0
4    1.0  0.0  3.0       0
```

```
import numpy as np

# Check missing values
df_heart = df_heart.replace('?', np.nan)
df_heart = df_heart.fillna(df_heart.mean(numeric_only=True))

# Convert target to binary (0 = No Disease, 1 = Disease)
df_heart['target'] = df_heart['target'].apply(lambda x: 1 if x > 0 else 0)

# Split data
from sklearn.model_selection import train_test_split
```

```
X_h = df_heart.drop('target', axis=1)
y_h = df_heart['target']

X_h_train, X_h_test, y_h_train, y_h_test = train_test_split(X_h, y_h, test_size=0.2, random_state=42)

print("Data Ready!")
print("Training size:", X_h_train.shape)
print("Testing size:", X_h_test.shape)
print()
print("Disease cases:", y_h.sum())
print("No Disease cases:", (y_h == 0).sum())
```

```
Data Ready!
Training size: (242, 13)
Testing size: (61, 13)

Disease cases: 139
No Disease cases: 164
```

```
# Fix NaN values properly
from sklearn.impute import SimpleImputer
import numpy as np

# Replace ? with NaN
df_heart = df_heart.replace('?', np.nan)

# Convert all to numeric
df_heart = df_heart.apply(pd.to_numeric, errors='coerce')

# Fix target column
df_heart['target'] = df_heart['target'].apply(lambda x: 1 if x > 0 else 0)

# Fill ALL missing values with mean
imputer = SimpleImputer(strategy='mean')
X_h = df_heart.drop('target', axis=1)
y_h = df_heart['target']

X_h = pd.DataFrame(imputer.fit_transform(X_h), columns=X_h.columns)

# Split data again
X_h_train, X_h_test, y_h_train, y_h_test = train_test_split(X_h, y_h, test_size=0.2, random_state=42)

print("Data Fixed!")
print("Training size:", X_h_train.shape)
print("Testing size:", X_h_test.shape)
print("Any NaN left?", X_h.isnull().sum().sum())
```

```
Data Fixed!
Training size: (242, 13)
Testing size: (61, 13)
Any NaN left? 0
```

```
# Train KNN Model on Heart Disease
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, f1_score, confusion_matrix

# Create KNN model
knn_h = KNeighborsClassifier(n_neighbors=5)

# Train model
knn_h.fit(X_h_train, y_h_train)

# Training Results
y_h_train_pred = knn_h.predict(X_h_train)
print("=== Training Results ===")
print("Accuracy:", round(accuracy_score(y_h_train, y_h_train_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y_h_train, y_h_train_pred) * 100, 2), "%")
print("F1 Score:", round(f1_score(y_h_train, y_h_train_pred) * 100, 2), "%")
print()

# Testing Results
y_h_test_pred = knn_h.predict(X_h_test)
print("=== Testing Results ===")
print("Accuracy:", round(accuracy_score(y_h_test, y_h_test_pred) * 100, 2), "%")
print("Precision:", round(precision_score(y_h_test, y_h_test_pred) * 100, 2), "%")
print("F1 Score:", round(f1_score(y_h_test, y_h_test_pred) * 100, 2), "%")
print()
```

```
# Confusion Matrix
print("=== Confusion Matrix ===")
print(confusion_matrix(y_h_test, y_h_test_pred))
```

```
=== Training Results ===
Accuracy: 75.62 %
Precision: 73.08 %
F1 Score: 72.04 %

=== Testing Results ===
Accuracy: 68.85 %
Precision: 74.07 %
F1 Score: 67.8 %

=== Confusion Matrix ===
[[22  7]
 [12 20]]
```

```
# Final Results Table
results = {
    'Dataset': ['Heart Disease', 'Heart Disease'],
    'Type': ['Training', 'Testing'],
    'Accuracy': ['75.62%', '68.85%'],
    'Precision': ['73.08%', '74.07%'],
    'F1 Score': ['72.04%', '67.80%']
}

df_results = pd.DataFrame(results)
print("=== Final Results Table ===")
print(df_results.to_string(index=False))
```

```
=== Final Results Table ===
      Dataset     Type Accuracy Precision F1 Score
Heart Disease Training   75.62%    73.08%   72.04%
Heart Disease  Testing   68.85%    74.07%   67.80%
```

```
# Simple Linear Regression on Heart Disease
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Create model
lr_h = LinearRegression()

# Train model
lr_h.fit(X_h_train, y_h_train)

# Training Results
y_h_train_pred_lr = lr_h.predict(X_h_train)
print("=== Training Results ===")
print("MSE:", round(mean_squared_error(y_h_train, y_h_train_pred_lr), 4))
print("R2 Score:", round(r2_score(y_h_train, y_h_train_pred_lr), 4))
print()

# Testing Results
y_h_test_pred_lr = lr_h.predict(X_h_test)
print("=== Testing Results ===")
print("MSE:", round(mean_squared_error(y_h_test, y_h_test_pred_lr), 4))
print("R2 Score:", round(r2_score(y_h_test, y_h_test_pred_lr), 4))
```

```
=== Training Results ===
MSE: 0.1195
R2 Score: 0.5155

=== Testing Results ===
MSE: 0.1099
R2 Score: 0.5593
```

```
# Final Results Table
results = {
    'Dataset': ['Heart Disease', 'Heart Disease'],
    'Type': ['Training', 'Testing'],
```